

The Gaming Room Application
CS 230 Project Software Design
Version 4.0

Table of Contents

The Gaming Room Application — Software Design 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
Domain Model 4
Evaluation 5
Recommendations 7

Document Revision History

Version	Date	Author	Comments
1.0	05/23/2026	Dan Berry	Completed software design documentation and updated UML / domain model.
2.0	06/05/2026	Dan Berry	Added platform evaluation and system recommendations for deployment across multiple operating systems.
3.0	06/20/2026	Dan Berry	Completed recommendations section and finalized platform deployment recommendations.
4.0	10/15/2026	Dan Berry	Prepared portfolio edition. Removed assignment-template instructions, reorganized the UML diagram, and refined document formatting.

Executive Summary

The purpose of this software design is to develop a web-based gaming application that allows users to create and manage games, teams, and players efficiently. The application is designed using object-oriented programming principles to provide a structured, maintainable, and scalable solution for the client. The system uses Java classes to organize game data and manage interactions between objects while ensuring that duplicate entries are not created.

The proposed solution uses a singleton design pattern within the *GameService* class to ensure that only one active service instance exists during program execution. This allows centralized management of all games, teams, and players within the application. The design also introduces a shared *Entity* base class to reduce duplicated code and improve consistency across the Game, *Team*, and *Player* classes through inheritance.

The application is intended to support a distributed web-based environment where multiple users may interact with the system. To support future expansion and maintenance, the software design focuses on modularity, reusability, and efficient data management. The use of inheritance, encapsulation, and centralized service management helps create a flexible architecture that can be expanded as additional gaming features are introduced.

Requirements

The Gaming Room application must support the creation and management of games, teams, and players in a distributed web-based environment. The application should ensure that duplicate games, teams, and players are not created while allowing users to access shared game data efficiently. The software must also support scalability, maintainability, and secure management of application data across multiple users and platforms.

Design Constraints

1. Singleton Service Requirement
   The application uses the singleton design pattern to maintain one *GameService* instance within a running application process. This provides centralized management of game data within that process. A distributed deployment involving multiple server instances would require additional coordination to maintain consistent shared data.

2. Unique Object Names
   Games, teams, and players must maintain unique names within the system. The application checks existing objects before creating new ones to prevent duplicates. This helps maintain data integrity and avoids confusion when retrieving objects from memory.

3. Distributed Web-Based Environment
   The game application is being designed for use in a distributed web environment. This creates constraints related to scalability, synchronization, and consistent access to shared data across multiple users or sessions. The proposed architecture must account for consistent access to shared game data across multiple application instances. The *GameService* class provides centralized management within an individual application process, while a distributed deployment would require additional mechanisms for coordinating shared data.

4. Object-Oriented Design Requirements
   The application must follow object-oriented programming principles including inheritance, encapsulation, and modularity. The addition of the *Entity* base class allows shared attributes and behaviors to be reused across multiple classes, reducing redundancy and improving maintainability.

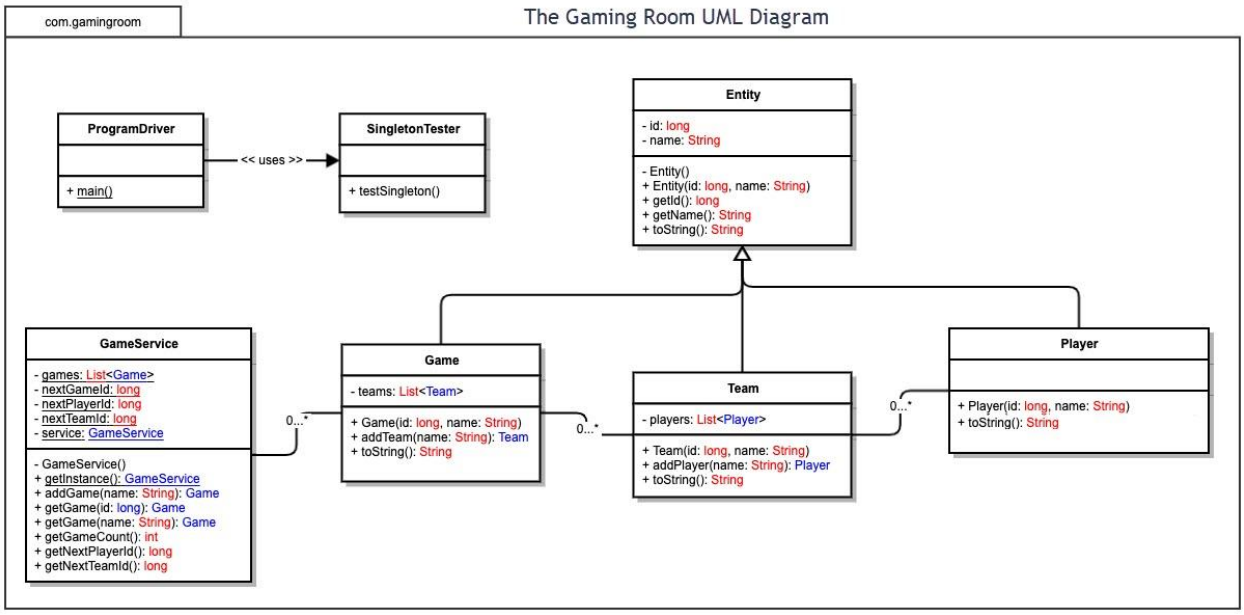
5. Expandability and Maintainability
   The software must be designed in a way that allows future expansion without requiring major redesign. By separating responsibilities into individual classes and using reusable methods, the system can more easily support additional features, such as expanded game management functionality or database integration in future development phases.

Domain Model

The UML class diagram represents the structure of the Draw It or Lose It application and shows how the classes interact within the system. The *Entity* class acts as a base class that provides common attributes such as *id* and *name* for the *Game*, *Team*, and *Player* classes. This demonstrates inheritance because the child classes reuse shared data and behaviors from the parent class rather than duplicating code.

The diagram also illustrates the relationships between the objects in the application. A game may contain one or more teams, and each team may contain multiple players. These relationships support the client's software requirements and help organize the application into manageable components. The *GameService* class manages active games and provides access to the application's game data. It implements the singleton pattern to maintain one service instance within a running application process.

Object-oriented programming principles such as inheritance, encapsulation, and abstraction are demonstrated throughout the design. Encapsulation is used by controlling access to object data through methods, while inheritance reduces duplicated code and improves maintainability. These design decisions help create a scalable and organized application structure for future development.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac servers can host web-based applications through Apache, Nginx, and Java application servers. MacOS provides a stable Unix-based environment and offers good security and reliability. However, Apple hardware is expensive compared to Linux or Windows servers and is not commonly used for large-scale web hosting environments. Licensing and hardware costs make Mac less attractive for hosting a web application expected to support thousands of concurrent players.	Linux is an excellent platform for hosting web-based applications because it is stable, secure, scalable, and widely used in enterprise environments. Linux supports Java, Apache, Nginx, MySQL, and numerous cloud-hosting platforms. Linux distributions such as Ubuntu Server offer free deployment options, while enterprise distributions such as Red Hat Enterprise Linux provide commercial subscriptions and support. The flexibility and efficiency of Linux make it the strongest candidate for hosting Draw It or Lose It.	Windows Server provides strong enterprise support and integrates well with Microsoft technologies. It offers graphical administration tools that may simplify management for some organizations. However, Windows Server licensing costs are significantly higher than Linux alternatives. Additional resources are often required to maintain Windows environments, making Linux a more cost-effective option for large-scale web hosting.	Mobile devices are not appropriate platforms for hosting a web-based gaming application. Mobile operating systems are designed primarily as client platforms rather than server platforms. Limitations involving performance, storage, scalability, and reliability make mobile devices unsuitable for supporting thousands of concurrent users.

Client Side	Supporting Mac clients primarily requires ensuring compatibility with Safari and other common browsers. Since the application will use responsive HTML delivered through a browser, development costs remain relatively low. Additional testing is necessary to verify consistent behavior across different browser versions and Mac hardware configurations.	Linux clients can access the application through modern web browsers such as Firefox and Chrome. Because Linux users may use a variety of distributions and desktop environments, additional browser compatibility testing is required. However, the browser-based approach eliminates the need to develop Linux-specific software.	Windows remains the most common desktop operating system and supports all major browsers. A browser-based solution reduces development complexity because no Windows-specific client installation is required. Testing should focus on browser compatibility and user interface responsiveness.	Mobile support requires responsive web design that automatically adapts to different screen sizes and touch-based interfaces. Additional development effort is needed to ensure compatibility with Android and iOS devices. However, using responsive web technologies avoids the cost of maintaining separate native applications for each mobile platform.
Development Tools	Development on Mac can use Java, HTML, CSS, JavaScript, Eclipse, IntelliJ IDEA, Visual Studio Code, and Git. Mac systems also provide access to Xcode if future native iOS development becomes necessary. Development costs are increased due to Apple hardware requirements.	Linux supports Java development through Eclipse, IntelliJ IDEA, Visual Studio Code, Maven, Git, Apache, Nginx, and MySQL. Most development tools are free and open source. Linux provides an efficient environment for server-side development and deployment.	Windows supports Java development through Eclipse, IntelliJ IDEA, Visual Studio Code, and Microsoft development tools. Most required development tools are available without additional licensing costs. Windows provides a familiar development environment for many software teams.	Mobile deployment requires HTML5, CSS3, JavaScript, responsive design frameworks, browser testing tools, and device emulators. Additional testing is required across Android and iOS devices to ensure a consistent user experience. Specialized mobile testing expertise may be needed depending on project requirements.

Recommendations

1. **Operating Platform**: I recommend Linux as the primary server operating platform for The Gaming Room. Linux provides excellent scalability, security, stability, and cost efficiency. The operating system is widely used in enterprise and cloud-hosting environments and supports the technologies required to host a distributed web-based gaming application. Linux also offers strong performance under heavy workloads while avoiding the licensing costs associated with commercial server operating systems. These advantages align directly with The Gaming Room's requirement to support thousands of concurrent users while maintaining long-term cost-effectiveness and system reliability.

2. **Operating Systems Architectures**: I recommend a Linux-based server architecture for The Gaming Room. Linux uses a layered architecture consisting of hardware, the kernel, system libraries, and user applications. This architecture separates system responsibilities and improves stability, scalability, and efficient resource management. This separation of components improves maintainability and allows individual layers to be updated without affecting the entire system. This is important for a distributed web-based gaming application.

   Linux also supports multiuser and multitasking environments, allowing thousands of concurrent users to access the application simultaneously. Because Draw It or Lose It will be delivered through a web browser, the server architecture can efficiently process client requests, manage game sessions, communicate with backend databases, and deliver content across multiple platforms. The modular design of Linux allows administrators to customize the operating system for specific workloads while maintaining reliability and performance as system demands grow.

3. **Storage Management**: Linux provides flexible and efficient storage management through modern file systems such as ext4 and XFS. These file systems support large amounts of data, efficient file access, journaling, and recovery mechanisms that help protect data in the event of a system failure. For Draw It or Lose It, storage will be required for user accounts, game sessions, image libraries, player statistics, and application logs.

   The application is expected to maintain approximately 200 image files at 8 MB each, requiring roughly 1.6 GB of storage for the image library alone (200 X 8MB = 1.6 GB). Because the image library may continue to grow over time, cloud-based storage should be considered as part of the overall storage strategy. Cloud storage provides scalability, redundancy, and accessibility while allowing administrators to expand storage capacity without significant infrastructure changes. Linux also supports scalable database solutions that can grow with the user base. Regular backups and redundant storage solutions should be implemented to minimize the risk of data loss and improve long-term system reliability.

4. **Memory Management**: Linux uses virtual memory management to efficiently allocate system resources among running processes. Memory is allocated as needed and reclaimed when applications no longer require it. This approach helps maximize system performance and allows the operating system to support many concurrent users without exhausting physical memory resources. For Draw It or Lose It, efficient memory management is important because the server will need to manage multiple active game sessions, player connections, and image-processing tasks simultaneously. Linux's memory management features help maintain responsiveness and stability as system demand increases.

5. **Distributed Systems and Networks**: Draw It or Lose It will operate as a distributed system in which users connect from various devices and locations through a network to a centralized server environment. Communication between clients and servers can be accomplished using standard web technologies such as HTTPS and RESTful web services. Because players may use Windows, Mac, Linux, Android, or iOS devices, the application should be designed using responsive web technologies to ensure consistent functionality across platforms.

   Because users may connect from desktop computers, laptops, tablets, and smartphones, the application must support a wide variety of screen sizes, operating systems, and browsers. Responsive web design allows a single application to adapt to different devices while providing a consistent user experience. This approach reduces development complexity and maintenance costs because only one code base must be maintained instead of creating separate applications for each platform.

   Network reliability, latency, connectivity interruptions, and server availability must all be considered when designing the system. Redundant servers, cloud hosting, load balancing, and monitoring tools can improve reliability and reduce the impact of outages. Shared session data and persistent game information would also require coordination between application instances to maintain consistency in a distributed environment. By using a distributed architecture and modern web technologies, The Gaming Room can provide reliable access to users regardless of the device or operating system they use.

6. **Security**: Security is a critical requirement for The Gaming Room because user information and game data must be protected. Linux provides a strong security foundation through user permissions, file ownership controls, and role-based access restrictions. Data transmitted between clients and servers should be protected using HTTPS and industry-standard encryption protocols. Strong authentication mechanisms, password policies, and regular software updates should be implemented to reduce security risks. System logs and monitoring tools should be used to detect suspicious activity and support incident investigations. By following the principle of least privilege and applying security best practices, The Gaming Room can better protect user accounts, application data, and server resources.